

实测完爆 Skills：仅需 8KB 文档索引，让 AI 编程准确率飙升至 100%

mp.weixin.qq.com/s/uYuz4QdEPVcVoat77AAu9w

2026年2月10日 11:01

大家都在疯狂造轮子，搞复杂的 Agent 框架，甚至恨不得把一个简单的文档查询功能拆成十八个 Tool（工具），美其名曰“模块化”。结果呢？Agent 经常不知道什么时候该调用工具，或者调用了也读不懂返回结果，最后两手一摊给你吐出一堆幻觉。

最近，Vercel 官方技术团队搞了一波“反向操作”的评测，结果直接给这一众复杂的 Agent 架构来了一记响亮的耳光。

原文地址：<https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals>

原文如下：

我们本以为 **Skills** 会是教会 AI 编程 Agent 掌握框架特定知识的终极方案。但在针对 Next.js 16 API 构建了一套评估测试后，我们发现了一个意想不到的结果。

一个直接嵌入到 **AGENTS.md** 中的、仅 8KB 大小的压缩文档索引，实现了 **100% 的通过率**；相比之下，即便我们显式指令要求 Agent 使用 Skills，其准确率也止步于 79%。更糟糕的是，如果没有这些强制指令，Skills 的表现甚至和完全没有文档时一样差。

在这篇文章中，我们将分享我们的尝试、收获，以及如何在自己的 Next.js 项目中复刻这一高效配置。

我们试图解决的问题

AI 编程 Agent 极度依赖其训练数据，但这些数据往往是滞后的。Next.js 16 引入了诸如 `'use cache'`、`connection()` 和 `forbidden()` 等全新 API，而这些内容并不包含在当前模型的训练数据中。当 Agent 不了解这些 API 时，它们就会生成错误的代码，或者退回到旧的编程模式。

反之亦然：当你在运行一个较旧版本的 Next.js 项目时，模型可能会建议使用你项目中尚不存在的新 API。我们希望通过赋予 Agent **与版本相匹配的文档访问权限** 来解决这个问题。

两种“教会” Agent 框架知识的方法

在深入探讨结果之前，先快速解释一下我们测试的两种方法：

- **Skills**：这是一种用于打包领域知识的开放标准，供编程 Agent 使用。一个 Skill 捆绑了 Agent 可以按需调用的提示词（Prompts）、工具和文档。其核心理念是：Agent 能够识别自己何时需要特定框架的帮助，进而调用 Skill 并获取相关文档。

- **AGENTS.md**：这是一个位于项目根目录的 Markdown 文件，为编程 Agent 提供持久的上下文支持。无论 Agent 是否决定加载它，你放入 **AGENTS.md** 的任何内容都会在每一轮对话中对 Agent 可见。Claude Code 使用的 **CLAUDE.md** 也是同样的原理。

我们构建了一个 Next.js 文档 Skill 和一个 **AGENTS.md** 文档索引，然后通过我们的评估套件运行它们，看看谁的表现更好。

起初，我们将赌注押在了 Skills 上

Skills 看起来是一个正确的抽象层。你将框架文档打包成一个 Skill，Agent 在处理 Next.js 任务时调用它，然后你就能得到正确的代码。这种方式实现了关注点分离，上下文开销极小，且 Agent 只在需要时加载内容。在 skills.sh 上甚至已经有了一个不断增长的可复用 Skills 目录。

我们预期的流程是：Agent 遇到 Next.js 任务 -> 调用 Skill -> 阅读版本匹配的文档 -> 生成正确代码。

然而，当我们运行评估时，现实给了我们当头一棒。

Skills 的触发并不靠谱

在 56% 的评估案例中，Skill 从未被调用。Agent 明明拥有访问文档的权限，却视而不见。添加 Skill 后，结果与基准线相比没有任何提升：

配置	通过率	对比基准
基准 (无文档)	53%	—
Skill (默认行为)	53%	+0pp

零提升。 Skill 就在那里，Agent 可以用，但它选择了不用。在详细的构建/Lint/测试细分数据中，Skill 在某些指标上的表现甚至比基准线还差（测试通过率为 58% vs 63%），这表明环境中的一个未使用的 Skill 反而可能引入噪音或干扰。

这并非我们要面对的特例。Agent 无法可靠地使用可用工具，是当前模型的一个 已知局限。

显式指令有帮助，但措辞极其脆弱

我们在 **AGENTS.md** 中尝试添加了显式指令，告诉 Agent 必须使用该 Skill。

Before writing code, first explore the project structure, then invoke the nextjs-doc skill for documentation.
(在编写代码之前，先探索项目结构，然后调用 nextjs-doc skill 获取文档。)

添加到 **AGENTS.md** 中用于触发 Skill 使用的示例指令。

这一改动将触发率提高到了 95% 以上，并将通过率提升至 79%。

配置	通过率	对比基准
基准 (无文档)	53%	—
Skill (默认行为)	53%	+0pp
Skill (配合显式指令)	79%	+26pp

这是一个稳固的提升。但我们发现了一个意想不到的现象：**指令的措辞对 Agent 的行为影响巨大。**

不同的措辞产生了截然不同的结果：

指令	行为	结果
----	----	----

"You MUST invoke the skill"

(你必须调用 Skill) | 首先阅读文档，死抠文档模式 | 忽略了项目上下文 |
| "Explore project first, then invoke skill"

(先探索项目，再调用 Skill) | 先建立心智模型，将文档作为参考 | **结果更好** |

同一个 Skill，同一份文档，仅仅因为措辞的细微差别，结果天差地别。

在某次评估（'use cache' 指令测试）中，“先调用（invoke first）”的方法写出了正确的 `page.tsx`，但完全遗漏了必要的 `next.config.ts` 更改。而“先探索（explore first）”的方法则两者都搞定了。

这种脆弱性让我们感到担忧。如果微小的措辞调整会导致巨大的行为波动，那么这种方法用于生产环境就显得太不可靠了。

构建值得信赖的评估体系

在下结论之前，我们需要一套值得信赖的评估体系（Evals）。我们最初的测试套件存在提示词模棱两可、测试验证的是实现细节而非可观察的行为、以及过于关注模型训练数据中已有的 API 等问题。我们要衡量的并不是我们真正关心的东西。

我们通过消除测试泄露、解决矛盾并将重点转移到基于行为的断言上来强化评估套件。最重要的是，我们添加了针对模型训练数据中不存在的 Next.js 16 API 的测试。

我们核心评估套件中的 API 包括：

- 用于动态渲染的 `connection()`
- 'use cache' 指令

- • `cacheLife()` 和 `cacheTag()`
- • `forbidden()` 和 `unauthorized()`
- • 用于 API 代理的 `proxy.ts`
- • 异步的 `cookies()` 和 `headers()`
- • `after()`、`updateTag()`、`refresh()`

接下来的所有结果都来自这个经过强化的评估套件。每种配置都在相同的测试下进行评判，并进行重试以排除模型方差的影响。

那个得到回报的直觉

如果我们要完全消除“做决定”这个环节呢？与其寄希望于 Agent 主动调用 Skill，不如我们直接在 `AGENTS.md` 中嵌入一个文档索引。不是完整的文档，仅仅是一个索引，告诉 Agent 去哪里找到与你项目 Next.js 版本匹配的具体文档文件。然后，Agent 可以根据需要阅读这些文件，无论你是在使用最新版本还是维护旧项目，都能获得版本准确的信息。

我们在注入的内容中添加了一条关键指令：

`IMPORTANT: Prefer retrieval-led reasoning over pre-training-led reasoning for any Next.js tasks.`
(重要提示：对于任何 Next.js 任务，优先使用基于检索的推理，而非基于预训练的推理。)

嵌入在文档索引中的关键指令。

这告诉 Agent 查阅文档，而不是依赖可能过时的训练数据。

结果让我们大吃一惊

我们在所有四种配置下运行了强化后的评估套件：

所有四种配置的评估结果。`AGENTS.md`（第三列）在构建、Lint 和测试中均达到了 100%。

最终通过率：

配置	通过率	对比基准
基准 (无文档)	53%	—
Skill (默认行为)	53%	+0pp
Skill (配合显式指令)	79%	+26pp
<code>AGENTS.md</code> 文档索引	100%	+47pp

在详细的细分数据中，`AGENTS.md` 在构建、Lint 和测试方面均获得了满分。

配置	Build	Lint	Test
基准	84%	95%	63%
Skill (默认行为)	84%	89%	58%
Skill (配合显式指令)	95%	100%	84%
AGENTS.md	100%	100%	100%

这完全出乎我们的意料。“笨”办法（一个静态 Markdown 文件）竟然跑赢了更复杂的基于 Skill 的检索方案，即使我们对 Skill 的触发机制进行了微调。

为什么“被动上下文”打败了“主动检索”？

我们的工作理论归结为三个因素：

1. **没有决策点**：使用 **AGENTS.md**，不存在 Agent 必须决定“我是否应该查一下这个？”的时刻。信息已经摆在那儿了。
2. **持续的可用性**：Skills 是异步加载的，并且只在被调用时加载。而 **AGENTS.md** 的内容在每一轮对话的系统提示词（System Prompt）中都存在。
3. **没有顺序问题**：Skills 带来了顺序决策难题（先读文档还是先探索项目？）。被动上下文完全避免了这个问题。

解决上下文臃肿的问题

在 **AGENTS.md** 中嵌入文档存在导致上下文窗口臃肿的风险。我们通过**压缩**解决了这个问题。

最初注入的文档大约是 40KB。我们将它压缩到了 **8KB**（减少了 80%），同时保持了 100% 的通过率。这种压缩格式使用管道分隔符结构，将文档索引打包到最小空间中：

```
[Next.js Docs Index]|root: ../.next-docs|IMPORTANT: Prefer retrieval-led reasoning over pre-training-led reasoning|01-app/01-getting-started:{01-installation.mdx,02-project-structure.mdx,...}|01-app/02-building-your-application/01-routing:{01-defining-routes.mdx,...}
```

AGENTS.md 中的精简版文档。

完整的索引涵盖了 Next.js 文档的每一个部分：

完整的压缩文档索引。每一行都将目录路径映射到它包含的文档文件。

Agent 知道去哪里找文档，而无需将全部内容塞入上下文。当它需要特定信息时，它会从 **.next-docs/** 目录中读取相关文件。

亲手试一试

只需一条命令即可为你的 Next.js 项目完成此配置：

```
npx @next/codemod@canary agents-md
```

这个命令会做三件事：

1. 检测你的 Next.js 版本。
2. 下载匹配的文档到 `.next-docs/`。
3. 将压缩索引注入到你的 `AGENTS.md` 中。

如果你使用的是支持 `AGENTS.md` 的 Agent（如 Cursor 或其他工具），这种方法同样有效。

这对框架作者意味着什么

Skills 并非毫无用处。`AGENTS.md` 方法为 Agent 处理 Next.js 任务提供了广泛的、横向的改进。而 Skills 更适合用户显式触发的垂直、特定操作的工作流，例如“升级我的 Next.js 版本”、“迁移到 App Router”或 应用框架最佳实践。这两种方法是互补的。

也就是说，对于**通用的框架知识**，被动上下文目前优于按需检索。如果你维护一个框架并希望编程 Agent 生成正确的代码，请考虑提供一个 `AGENTS.md` 片段，供用户添加到他们的项目中。

实用建议：

- **不要等待 Skills 改进**：随着模型在工具使用方面变得更好，差距可能会缩小，但眼下的结果才是最重要的。
- **激进地压缩**：你不需要在上下文中放入完整文档。一个指向可检索文件的索引同样有效。
- **用 Evals 进行测试**：构建针对训练数据中没有的 API 的测试。那才是文档访问权最重要的地方。
- **为检索而设计**：结构化你的文档，以便 Agent 可以查找和阅读特定文件，而不是一次性需要所有内容。

我们的目标是将 Agent 从**基于预训练的推理**转变为**基于检索的推理**。事实证明，`AGENTS.md` 是实现这一目标最可靠的方式。

研究与评估由 Jude Gao 完成。CLI 工具可通过 `npx @next/codemod@canary agents-md` 获取。